

Bash Mastery — Printable Notes

bash-learning curriculum

2026-06-04

Table of Contents

1. Module 1 — Bash Basics
2. Module 2 — Conditionals
3. Module 3 — Loops
4. Module 4 — Functions
5. Module 5 — I/O & Redirection
6. Module 6 — Arrays
7. Module 7 — String Manipulation & Regex
8. Module 8 — Processes, Signals, Traps
9. Module 9 — Text Processing (grep / sed / awk)
10. Module 10 — Error Handling & Debugging
11. Module 11 — Advanced Patterns

Module 1 — Bash Basics

Bash Basics Study Notes

Shebang

- Put the shebang on the first line of the script.
- `#!/bin/bash` tells the system to run the file with Bash.
- Common workflow:

```
chmod +x script.sh
./script.sh
```

Variables & quoting basics

- Create variables with `name="Sai"`.
- Do not put spaces around `=`.
- Read a value with `$name` or `${name}`.
- Use `${name}` when text touches the variable name.
- Quote variable expansions to keep spaces safe.

```
name="Hitesh"
echo "$name"
echo "Sai${name}"
```

Positional parameters reference table

Syntax	Meaning	Example
--------	---------	---------

\$0	Script name/path used to start the script	./Lesson1.sh
\$1	First argument	John
\$2	Second argument	Michael
\$#	Number of arguments	2
\$@	All arguments, preserved as separate items when quoted	John Michael
\$*	All arguments, often joined into one string when quoted	John Michael

```
#!/bin/bash
echo "Script: $0"
echo "First: $1"
echo "Second: $2"
echo "Count: $#"
```

```
echo "All: $@"
```

read cheatsheet

- Basic form: read name
- Prompt + read in one line: read -p "Enter your name: " name
- Do not write read \$name.
- read \$name expands first and can target the wrong variable.

```
echo "Enter your city:"
read city
echo "City: $city"
```

```
read -p "Enter your age: " age
echo "Age: $age"
```

Common pitfalls

- **Wrong:** name = "Sai"
- **Right:** name="Sai"
- **Wrong:** read \$name
- **Right:** read name
- **Wrong target:** echo "\$name_file"
- **Clearer:** echo "\${name}_file"
- Unquoted variables can break when values contain spaces.

```
file_name="my notes.txt"
echo "$file_name"
```

Key takeaways

- Use #!/bin/bash on line 1.
- Variable assignment has no spaces around =.
- Use \$1, \$2, \$@, and \$# for CLI input.
- Use read name or read -p "Prompt" name for user input.
- Prefer \${name} when joining variables with other text.
- Quote variables unless you specifically need word splitting.

Module 2 — Conditionals

Bash Conditionals Notes

if / elif / else syntax

Use if when your script must choose between conditions.

```
if [ "$marks" -ge 90 ]; then
    echo "Grade: A"
elif [ "$marks" -ge 60 ]; then
    echo "Grade: B"
else
    echo "Grade: C"
fi
```

Quick rules: - then starts the command block. - elif adds another condition. - else is the fallback. - fi closes the statement.

Test operators table

Numeric tests

Operator	Meaning
-eq	equal to
-ne	not equal to
-lt	less than
-le	less than or equal to
-gt	greater than
-ge	greater than or equal to

String tests

Operator	Meaning
= or ==	strings are equal
!=	strings are not equal
<	left string comes before right string alphabetically
>	left string comes after right string alphabetically
-z	string is empty
-n	string is not empty

File tests

Operator	Meaning
-f	path exists and is a regular file
-d	path exists and is a directory
-e	path exists
-r	path is readable
-w	path is writable
-x	path is executable
-s	file exists and is not empty
-L	path is a symbolic link

[] vs [[]]

[...] is the traditional POSIX test form. [[...]] is Bash-specific and more powerful.

```
file_name="notes.txt"

if [ "$file_name" = "notes.txt" ]; then
    echo "Exact match"
```

```

fi

if [[ $file_name == *.txt ]]; then
    echo "Pattern match"
fi

```

[[]] superpowers

Use [[...]] when you want extra Bash features.

- Supports pattern matching with ==.
- Supports regex matching with =~.
- Lets you combine checks with && and ||.
- Avoids word splitting and pathname expansion in a safer way.

```

text="hello world"
file_name="report.txt"

if [[ $file_name == *.txt && -n $text ]]; then
    echo "Text file with non-empty text"
fi

email="student42@example.com"
if [[ $email =~ ^[a-z0-9]+\@example\.com$ ]]; then
    echo "Valid example.com email"
fi

```

case statement

Use case when one value can match several patterns.

```

action="reload"

case "$action" in
    start|run|up)
        echo "Start service"
        ;;
    stop|down)
        echo "Stop service"
        ;;
    restart|reload)
        echo "Reload service"
        ;;
    *)
        echo "Unknown action"
        ;;
esac

```

Useful reminders: - Separate alternate patterns with |. - End each branch with ;; in normal usage. - *) is the default branch. - Close the block with esac.

Interactive menu pattern

A common beginner pattern is read + case.

```

echo "1) Show date"
echo "2) Show current directory"
echo "3) Exit"
read -r -p "Choose an option: " choice

case "$choice" in
    1) echo "Date: $(date)" ;;
    2) echo "Directory: $(pwd)" ;;
    3) echo "Goodbye" ;;
    *) echo "Invalid choice" ;;
esac

```

Common pitfalls

- **Unquoted variables in []:** [\$name = hello] can break if \$name is empty or contains spaces.
- **Using = or == for numbers:** ["\$a" = "\$b"] and ["\$a" == "\$b"] compare text, not numeric value; use -eq.
- **Mixing = and ==:** = is the portable string operator for [...]; == is commonly used inside [[...]].
- **Using < or > inside []:** they may be treated as redirection; prefer [[...]].
- **Missing fi:** every if block must end with fi.
- **Missing esac:** every case block must end with esac.

```
name="hello world"
a=10
b=2

## Safe string test
if [ "$name" = "hello world" ]; then
    echo "match"
fi

## Safe numeric test
if [ "$a" -gt "$b" ]; then
    echo "a is greater"
fi
```

Quick checklist

- Quote variables in [...].
- Use -eq, -lt, -gt for numbers.
- Use [[...]] for patterns, regex, and combined logic.
- Use file flags like -f, -d, and -e for path checks.
- Use case for many fixed choices.

Module 3 — Loops

Module 03 — Printable Notes

for (list form)

Use this form when you already have a list of values.

```
for fruit in apple banana cherry
do
    echo "$fruit"
done
```

- Good for simple fixed lists
- Also useful with globs such as *.sh
- Quote variables when printing: "\$fruit"

for (C-style)

Use this when you need a numeric counter.

```
for ((i=1; i<=5; i++))
do
    echo "$i"
done
```

Structure:

```

for ((initialization; condition; increment))
do
    commands
done

```

while

A while loop runs as long as its condition is true.

```

count=1
while [ "$count" -le 3 ]
do
    echo "$count"
    ((count++))
done

```

Common uses: - Counters - Repeating work until a condition changes - Reading input or files line by line

until

An until loop runs until its condition becomes true.

```

count=1
until [ "$count" -gt 3 ]
do
    echo "$count"
    ((count++))
done

```

Think of it as the opposite of while.

break and continue

- break exits the loop immediately
- continue skips the rest of the current iteration

```

for n in 1 2 3 4 5
do
    [ "$n" -eq 2 ] && continue
    [ "$n" -eq 4 ] && break
    echo "$n"
done

```

Output:

```

1
3

```

Reading a file line by line

Canonical safe pattern:

```

while IFS= read -r line
do
    echo "$line"
done < file.txt

```

Why this pattern? - IFS= preserves leading/trailing whitespace - read -r prevents backslash escaping - < file.txt avoids unnecessary command substitution

Reading colon-separated files such as /etc/passwd:

```

while IFS=: read -r user ____ shell

```

```
do
  echo "$user -> $shell"
done < /etc/passwd
```

Looping over arrays vs command substitution

Prefer arrays when values may contain spaces.

Safe array loop

```
files=("one file.txt" "two file.txt" "notes.md")
for f in "${files[@]}"
do
  echo "$f"
done
```

Risky command substitution

```
for f in $(ls)
do
  echo "$f"
done
```

Why risky? - Word splitting breaks filenames with spaces - Output from `ls` is for display, not robust parsing

Better options:

```
for f in *
do
  [ -e "$f" ] || continue
  echo "$f"
done
```

or

```
while IFS= read -r file
do
  echo "$file"
done < file-list.txt
```

Common pitfalls

1. Unquoted variables

```
echo $file # risky
echo "$file" # safe
```

2. Using `for f in $(ls)`

Bad for filenames with spaces, tabs, or newlines.

3. Forgetting to update the loop condition

This can cause an infinite loop.

```
count=1
while [ "$count" -le 3 ]
do
  echo "$count"
  # missing ((count++))
done
```

4. Misusing IFS

Changing IFS globally can affect later splitting behavior. Prefer setting it for a single command:

```
while IFS= read -r line
do
    echo "$line"
done < file.txt
```

5. Reading files with for line in \$(cat file)

This splits on whitespace, not lines. Use `while IFS= read -r line` instead.

Quick comparison

Loop type	Best use
for item in list	Fixed lists, globs, array values
for ((...))	Numeric counters
while	Repeat while condition is true
until	Repeat while condition is false

Memory tips

- Use `for ((...))` for numbers
- Use `while` for counters and files
- Use `until` when you want “keep going until this becomes true”
- Use `break` to stop early
- Use `continue` to skip one iteration
- Use `while IFS= read -r line; do ...; done < file` for safe line-by-line reading

Module 4 — Functions

Module 04: Functions — Notes

Defining functions

Functions let you group commands under a reusable name.

```
say_hello() {
    echo "Hello"
}

function say_hi {
    echo "Hi"
}
```

- Prefer `name() { ... }` for everyday bash scripts.
- `function name { ... }` works in bash, but it is less portable to plain sh.
- Call a function by writing its name: `say_hello`

Arguments

Functions receive positional parameters just like scripts do.

```
show_args() {
    echo "first: $1"
```



```

    echo "all: $@"
    echo "count: $#"
```

script name is still: \$0"

```

}
```

```
show_args apple banana
```

Inside the function: - \$1, \$2, ... = function arguments - \$@ = all function arguments - \$# = number of function arguments - \$0 = script name, not the function name

Return vs Output

In bash, return sets an **exit status**, not a general data value.

```

is_ok() {
    return 0
}
```

- 0 usually means success
- non-zero usually means failure or a special condition
- valid range is 0-255

If you want the function to give back actual data, print it:

```

make_name() {
    echo "$1 $2"
}

full_name="$(make_name Ada Lovelace)"
echo "$full_name"
```

Local scope

Variables are global by default in bash.

```

name="outside"

change_it() {
    name="changed"
}
```

That can cause bugs because the function modifies the caller's variable. Use local when the variable only belongs inside the function.

```

safe_change() {
    local name="inside"
    echo "$name"
}
```

Why it matters: - avoids accidental overwrites - makes functions easier to reason about - reduces surprising side effects

Recursion

A recursive function calls itself.

```

factorial() {
    local n="$1"
    if (( n <= 1 )); then
        echo 1
        return 0
    fi

    local smaller
    smaller="$(factorial $((n - 1)))"
```

```
    echo $((n * smaller))
}
```

Use recursion carefully in shell scripts. It is great for learning, but simple loops are often easier to read and debug.

Common pitfalls

1) Forgetting local

```
count=10
bump() {
    count=99
}
```

After calling bump, the outer count becomes 99.

2) Trying to return big numbers

```
myfunc() {
    return 300
}
```

Exit statuses are limited to 0-255, so large values do not behave like normal return values.

3) Expecting return to act like other languages

```
add() {
    return $(( $1 + $2 ))
}
```

This sets an exit status, not a printable sum. For data, use echo:

```
add() {
    echo $(( $1 + $2 ))
}
```

4) Modifying caller variables by mistake

```
message="hello"
show() {
    message="bye"
}
```

Use local message="bye" if the change should stay inside the function.

Quick pattern to remember

```
myfunc() {
    local arg1="$1"
    local arg2="${2:-default}"

    if [ -z "$arg1" ]; then
        return 1
    fi

    echo "$arg2 $arg1"
}

result="$(myfunc world Hello)"
```

- Use parameters for input
- Use local for temporary variables
- Use return for status

- Use echo/printf for output

Module 5 — I/O & Redirection

Module 05 Notes: I/O and Redirection

File Descriptor Table

FD	Name	Meaning
0	stdin	Standard input: where a command reads from
1	stdout	Standard output: normal command output
2	stderr	Standard error: error messages

Redirection Operators Table

Operator	Meaning	Example
>	Redirect stdout and overwrite target file	echo "hi" > out.txt
>>	Redirect stdout and append to target file	echo "hi" >> out.txt
2>	Redirect stderr only	ls missing 2> errors.log
2>>	Append stderr only	cmd 2>> errors.log
2>&1	Point stderr to wherever stdout currently goes	cmd > all.log 2>&1
&>	Redirect stdout and stderr together (bash shortcut)	cmd &> all.log
<	Read stdin from a file	wc -l < names.txt
<<<	Feed a single string to stdin (here-string)	read -r name <<< "Hitesh"
<<EOF	Feed a multi-line block to stdin (here-document)	cat <<EOF
<<-EOF	Here-document that strips leading tabs	cat <<-EOF
	Pipe stdout of one command into stdin of another	ls wc -l
tee	Copy stdin to stdout and a file	make tee build.log
<(cmd)	Expose command output as a readable pseudo-file	diff <(sort a) <(sort b)
>(cmd)	Expose a writable pseudo-file that sends data into a command	echo hi > >(tr a-z A-Z)
exec > file 2>&1	Redirect the shell/script itself from that point onward	exec > out.log 2>&1

Here-Docs and Here-Strings

Here-string

Use `<<<` when you have one short string and want to feed it into `stdin`.

```
read -r word <<< "hello"
```

Here-document

Use a here-document when you need multiple lines.

```
cat <<'EOF'
name=app
port=8080
EOF
```

Indented here-document

Use `<<-EOF` if you want to indent the body with **tabs** for readability. Spaces are not stripped.

```
cat <<-'EOF'
    line one
    line two
EOF
```

Process Substitution

Process substitution lets command output behave like a file path.

```
diff <(sort a.txt) <(sort b.txt)
```

That is often cleaner than creating temporary sorted files first.

You can also send output into a command with `>(cmd)`:

```
printf 'hello\n' > >(tr '[:lower:]' '[:upper:]')
```

Pipes and the Subshell Gotcha

Pipes connect `stdout` from the left command to `stdin` of the right command.

```
printf 'a\nb\n' | wc -l
```

A common bash gotcha:

```
count=0
printf 'a\nb\n' | while read -r line; do
    count=$((count+1))
done
echo "$count"
```

In many cases the while loop runs in a **subshell**, so `count` outside the loop stays unchanged.

Safer pattern when you need the variable afterward:

```
count=0
while read -r line; do
    count=$((count+1))
done <<(printf 'a\nb\n')
echo "$count"
```

Common Pitfalls

1. **Order matters**
 - `cmd >file 2>&1` redirects both stdout and stderr to file.
 - `cmd 2>&1 >file` usually leaves stderr going to the terminal.
2. **> overwrites**
 - Use `>>` if you want to keep existing content.
3. **<<-EOF strips tabs, not spaces**
 - If indentation is done with spaces, they stay in the output.
4. **Pipes may create subshells**
 - Variables changed inside a piped loop may not survive outside.
5. **2>/dev/null hides errors**
 - Helpful for noisy commands, but dangerous if it hides something important.
6. **Quote filenames in redirections and commands**
 - Example: `> "$log_file"` is safer than `> $log_file`.

Module 6 — Arrays

Module 06 Notes: Bash Arrays

Indexed arrays cheatsheet

Creation

```
arr=(a b c)
arr[3]=d
arr+=(e f)
```

Access

```
${arr[0]}    # first element
${arr[@]}    # all elements
${arr[*]}    # all elements
${!arr[@]}   # assigned indices
${#arr[@]}   # number of assigned elements
```

Iteration

```
for x in "${arr[@]"; do
    echo "$x"
done
```

Slicing

```
${arr[@]:start:count}
${arr[@]:1:2}
${arr[@]:2:3}
```

Associative arrays

```
declare -A map
map[name]="bash"
map[level]="intermediate"

${map[name]}    # access one value
${!map[@]}      # all keys
${#map[@]}      # number of keys
```

```
for key in "${!map[@]}; do
    echo "$key => ${map[$key]}"
done
```

Quoting matrix

Assume:

```
arr=("a b" "c")
```

Expression	Result
"\${arr[@]}"	Expands to two words : a b and c
"\${arr[*]}"	Expands to one word : a b c
\${arr[@]}	Unquoted; subject to word splitting and globbing
\${arr[*]}	Unquoted; subject to word splitting and globbing
printf '%s\n' "\${arr[@]}"	Safely prints each element on its own line

Common pitfalls

- Forgetting quotes around "\${arr[@]}" when elements may contain spaces.
- Expecting unset 'arr[1]' to shift later elements left automatically.
- Confusing \${#arr[@]} (element count) with string-length expansions.
- Using associative arrays without declare -A.
- Assuming associative arrays preserve insertion order; do not rely on key order.
- Passing arrays to functions by value instead of by name when you want generic helpers.

Module 7 — String Manipulation & Regex

Module 07 Notes: String Manipulation and Regex

Parameter Expansion Cheatsheet

Expression	Behavior	Example
\${#str}	Length of string	name="bash" -> 4
\${str:offset:length}	Slice substring	word="scripting" + \${word:0:6} -> script
\${str:-3}	Last 3 characters (negative offset needs a space)	word="bash" -> ash
\${str^^}	Uppercase all letters	bash -> BASH
\${str,,}	Lowercase all letters	BaSh -> bash
\${str^}	Uppercase first character	bash -> Bash
\${str,}	Lowercase first character	Bash -> bash
\${var#pattern}	Remove shortest	path="/a/b/c" ->

	matching prefix	<code>\${path#*/} = a/b/c</code>
<code>\${var##pattern}</code>	Remove longest matching prefix	<code>path="/a/b/c" -> \${path##*/} = c</code>
<code>\${var%pattern}</code>	Remove shortest matching suffix	<code>file="a.tar.gz" -> \${file%.*} = a.tar</code>
<code>\${var%%pattern}</code>	Remove longest matching suffix	<code>file="a.tar.gz" -> \${file%%.*} = a</code>
<code>\${var/old/new}</code>	Replace first match	<code>foo foo -> bar foo</code>
<code>\${var//old/new}</code>	Replace all matches	<code>foo foo -> bar bar</code>
<code>\${var/#old/new}</code>	Replace only at the start	<code>hello world -> hi world</code>
<code>\${var/%old/new}</code>	Replace only at the end	<code>report.txt -> report.md</code>
<code>\${var:-default}</code>	Use default if unset or empty	<code>unset x -> \${x:-guest} = guest</code>
<code>\${var:=default}</code>	Use and assign default if unset or empty	<code>unset x -> \${x:=guest} sets x=guest</code>
<code>\${var:?message}</code>	Error if unset or empty	<code>: "\${x:?x required}"</code>
<code>\${var:+alt}</code>	Use alternate text if set and non-empty	<code>x=1 -> \${x:+yes} = yes</code>

Case Conversion

- Requires **bash 4+**.
- `${text^^}` uppercases all matching letters.
- `${text,,}` lowercases all matching letters.
- `${text^}` changes only the first character to uppercase.
- `${text,}` changes only the first character to lowercase.
- Useful for normalization, headings, tags, and simple title-style output.

Pattern Matching vs Regex

Topic	Pattern Matching	Regex
Used by	Parameter expansion, case, <code>[[\$x == pattern]]</code>	<code>[[\$x =~ regex]]</code>
Syntax style	Globs like <code>*</code> , <code>?</code> , <code>[abc]</code>	Regex tokens like <code>^</code> , <code>\$</code> , <code>+</code> , <code>()</code> , <code>{2,}</code>
Quoting rule	Usually quote variables, not patterns you want expanded as patterns	Do not quote the right side if you want real regex matching
Example	<code>\${file##*/}</code> removes up to the last slash	<code>[[\$email =~ ^.+@.+\.+\$]]</code>

printf Formats

Format	Meaning	Example Result
<code>%s</code>	String	bash
<code>%d</code>	Integer	42
<code>%05d</code>	Integer, width 5, zero-padded	00042
<code>%.2f</code>	Floating-point with 2 decimals	3.14
<code>%q</code>	Shell-escaped string	two\ words

Common Pitfalls

1. **Negative substring offsets need a space:** write `${var: -3}`, not `${var:-3}`.
2. **#/##/%/%% use glob patterns, not regex.**
3. **/ replaces once, // replaces everywhere.**
4. **:- does not assign; := does assign.**
5. **\${var:?msg}** can exit the current shell/script if the variable is missing.
6. **For regex, do not quote the right side of =~** if you want bash to interpret regex syntax.
7. **Use read -r** unless you specifically want backslashes treated as escapes.
8. **Use printf instead of echo** when you need reliable formatting.

Module 8 — Processes, Signals, Traps

Module 08 Notes — Processes, Signals, and Traps

1. Process model

- Every running command is a process.
- A shell script runs inside a shell process.
- Foreground processes keep the shell busy until they finish.
- Background processes are started with `&` so the shell can continue immediately.
- `jobs` lists jobs started from the current shell session.
- `wait` pauses until one or more background jobs finish.
- `wait -n` pauses until the next background job finishes.
- `fg` and `bg` are mainly interactive-shell tools for resuming jobs in the foreground or background.

2. PID variables

- `$$` — PID of the current shell.
- `$!` — PID of the most recently started background job.
- `$PPID` — PID of the parent process.

Example:

```
echo "shell pid: $$"
sleep 5 &
echo "last bg pid: $!"
echo "parent pid: $PPID"
```

3. Signals table

Number	Name	Default action	Can trap?	Typical meaning
1	SIGHUP	Terminate	Yes	Terminal closed; often reused as reload
2	SIGINT	Terminate	Yes	Keyboard interrupt

				(Ctrl-C)
9	SIGKILL	Terminate immediately	No	Force-kill; cannot be caught or ignored
10	SIGUSR1	Terminate	Yes	User-defined application signal
12	SIGUSR2	Terminate	Yes	User-defined application signal
15	SIGTERM	Terminate	Yes	Polite request to stop cleanly

Notes: - Prefer SIGTERM before SIGKILL. - SIGKILL is a last resort because the process gets no chance to clean up. - Signal numbers can vary across some Unix systems, but the names are stable and easier to remember.

4. trap recipes

Cleanup on exit

```
cleanup() {
    rm -rf "$temp_dir"
}

trap 'cleanup' EXIT INT TERM
```

Use this when you create temp files, lock files, sockets, or directories that must be removed.

Retry wrapper

```
retry() {
    local attempt
    for attempt in 1 2 3; do
        "$@" && return 0
        sleep "$attempt"
    done
    return 1
}
```

Use retry when a command may fail temporarily, such as a network call or a transient lock.

Ignore a signal

```
trap '' INT
```

This tells the shell to ignore SIGINT. Use sparingly because ignoring Ctrl-C can be frustrating for users.

5. Parallel jobs pattern

A simple pattern for launching independent work in parallel:

```
work() {
    local item="$1"
    echo "starting $item"
```

```

sleep 1
echo "done $item"
}

for i in 1 2 3; do
    work "$i" &
done

wait

```

Key idea: - & starts each job in the background. - wait with no PID waits for all of them.

When you need per-job status, store the PIDs:

```

pids=()
for i in 1 2 3; do
    work "$i" &
    pids+=("$!")
done

for pid in "${pids[@]}; do
    wait "$pid"
done

```

6. Common pitfalls

- Forgetting that (...) uses a subshell, so variable changes do not come back out.
- Forgetting the spaces or trailing semicolon in { ...; }.
- Using SIGKILL first instead of trying SIGTERM.
- Setting a trap before the variable it uses is defined.
- Creating temp files but forgetting cleanup.
- Assuming jobs, fg, and bg behave the same way in non-interactive scripts as they do in an interactive terminal.
- Launching background jobs and forgetting wait, which can make a script exit early.
- Not checking timeout exit codes; 124 usually means the command exceeded the limit.

Module 9 — Text Processing (grep / sed / awk)

Module 09 Notes: Text Processing

Printable quick-reference notes for grep, sed, awk, and related tools.

grep cheatsheet

Basic form

```
grep 'pattern' file
```

Common flags

```

grep -i 'error' file    # case-insensitive
grep -v 'INFO' file     # invert match
grep -n 'TODO' file     # show line numbers
grep -r 'main' src/     # recursive search
grep -E 'cat|dog' file  # extended regex

```

```
grep -F 'a+b' file      # fixed string, no regex parsing
grep -c 'ERROR' file    # count matching lines
grep -l 'TODO' *.md     # print filenames with matches
grep -oE '[0-9]+' file  # print only the matched text
```

Common regex examples

```
grep -E '^ [A-Z]' file  # line starts with uppercase
grep -E '[0-9]{4}' file  # 4-digit number
grep -oE '[^ ]+@[^ ]+' file  # rough email extraction
```

sed cheatsheet

Basic substitution

```
sed 's/old/new' file    # first match per line
sed 's/old/new/g' file  # all matches per line
sed 's/foo/bar/2' file  # second match per line
sed -n 's/foo/bar/p' file # print only lines where substitution happened
```

Addressing and deletion

```
sed '1,5s/apple/orange/g' file # apply only to lines 1 through 5
sed '/DEBUG/d' file            # delete matching lines
sed '/start/,/end/d' file      # delete a range of lines
```

In-place editing

```
sed -i 's/TODO/DONE/g' notes.md
```

Portability note

- GNU sed (Linux): sed -i 's/a/b/' file
- BSD sed (macOS): sed -i '' 's/a/b/' file

Multiple commands

```
sed -e 's/error/ERROR/g' -e '/DEBUG/d' app.log
```

awk one-liners cheatsheet

Basic fields

```
awk '{print $0}' file    # whole line
awk '{print $1}' file    # first field
awk '{print $NF}' file   # last field
awk '{print NR, $0}' file # line number + line
```

Custom field separator

```
awk -F ',' '{print $1, $3}' users.csv
```

BEGIN / END

```
awk 'BEGIN {print "Start"} {print $1} END {print "Done"}' file
```

Filtering

```
awk '$3 > 100' file
awk -F ',' '$3 >= 1000 {print $1}' /etc/passwd
```

Sum a column

```
awk '{sum += $2} END {print sum}' numbers.txt
awk -F ',' 'NR > 1 {sum += $3} END {print sum}' report.csv
```

printf formatting

```
awk '{printf "%-10s %5d\n", $1, $2}' file
```

FS vs OFS

- FS: input field separator
- OFS: output field separator

Example:

```
awk 'BEGIN {FS=","; OFS=" | "} {print $1, $2}' file.csv
```

sort / uniq / cut / tr quick ref

cut

```
cut -d ',' -f1,3 users.csv
cut -c1-5 file
```

sort

```
sort file           # alphabetical
sort -n numbers.txt # numeric
sort -r file        # reverse
sort -u file         # unique lines after sorting
sort -nu numbers.txt # numeric sort + unique
sort -t ',' -k3,3nr report.csv
```

uniq

```
sort words.txt | uniq
sort words.txt | uniq -c
sort words.txt | uniq -d
```

tr

```
tr '[:upper:]' '[:lower:]' < file
tr -d ',' < file
tr -s ' ' < file
```

wc

```
wc file
wc -l file
wc -w file
wc -c file
```

head / tail

```
head -5 file
tail -10 file
tail -f app.log
```

Pipeline patterns

Top 5 most frequent words

```
tr '[:upper:]' '[:lower:]' < paragraph.txt \  
| tr -cs '[:alpha:]' '\n' \  
| sort \  
| uniq -c \  
| sort -rn \  
| head -5
```

Find errors in logs

```
grep -i 'error' app.log | tail -20
```

Extract a CSV column and sort it

```
cut -d ',' -f2 users.csv | sort | uniq
```

Count users with UID >= 1000

```
awk -F ':' '$3 >= 1000 {count++} END {print count}' /etc/passwd
```

Safe filename handling

```
find . -type f -print0 | xargs -0 ls -la
```

Common pitfalls

1. sed -i portability

- Linux usually uses GNU sed.
- macOS usually uses BSD sed.
- sed -i syntax is different between them.

2. GNU vs BSD differences

- Some flags and regex behaviors differ.
- If a command works on Linux but not macOS, check the system's man page.

3. uniq only removes adjacent duplicates

```
uniq words.txt
```

This only works as expected if identical lines are already next to each other, so often you want:

```
sort words.txt | uniq
```

4. cat file | grep ... is usually unnecessary

Prefer:

```
grep 'word' file
```

instead of:

```
cat file | grep 'word'
```

This is often called a UUOC: useless use of cat.

5. xargs without -0 can break on spaces

Prefer:

```
find . -type f -print0 | xargs -0 ls -la
```

when filenames may contain spaces or unusual characters.

Module 10 — Error Handling & Debugging

Module 10 Notes — Error Handling & Debugging

Printable reference for writing safer bash scripts.

1) set options quick table

Option	Long form	Meaning	Why it helps	Common caveat
-e	errexit	Exit when a simple command fails	Stops many scripts from continuing after an unexpected failure	Tricky in if, &&, , subshells, command substitutions, and other control-flow contexts
-u	nounset	Treat unset variables as errors	Catches typos like <code>\$usre_name</code> early	Can break scripts that assume missing variables expand to empty strings
-o pipefail	pipefail	Pipeline fails if any command fails	Prevents hidden failures on the left side of a pipeline	Some commands in pipelines use non-zero status for normal control flow
-x	xtrace	Print commands before executing them	Great for debugging and teaching	Very noisy; can leak secrets if enabled around sensitive values
-E	errtrace	Inherit ERR trap into functions/subshells more consistently	Makes central error reporting more useful	Still not a full replacement for explicit error handling

2) Standard script header

A common “safe by default” bash header:

```
#!/bin/bash
set -euo pipefail
IFS=$'\n\t'
```

Why this header is popular

- `#!/bin/bash` ensures the script runs with bash.
- `set -euo pipefail` enables a practical strict mode.
- `IFS=$'\n\t'` removes space from the default word-splitting characters, which reduces accidental splitting of filenames like `My File.txt`.

Important note about IFS

IFS controls how unquoted expansions split into words. Changing it does **not** eliminate the need to quote variables. The safest habit is still:

```
echo "$file"
cp -- "$src" "$dst"
```

3) Error-handling patterns

Pattern A: fail fast with die()

```
die() { echo "ERROR: $*" >&2; exit 1; }
```

Use it for unrecoverable errors such as bad arguments, missing files, or missing dependencies.

Pattern B: validate arguments at the top

```
[[ $# -ge 2 ]] || die "usage: $0 SOURCE DEST"
```

Validate early so the rest of the script can assume required inputs exist.

Pattern C: check required commands once

```
require_cmd() {
  command -v "$1" >/dev/null 2>&1 || die "required command not found: $1"
}

require_cmd curl
require_cmd jq
```

Pattern D: explicit handling for expected failures

```
if ! grep -q "$needle" "$file"; then
  echo "Pattern not found"
fi
```

Use explicit handling when a non-zero status is part of normal logic.

Pattern E: one-line fallback

```
mkdir -p "$dir" || die "could not create directory: $dir"
```

Pattern F: centralized logging with ERR

```
report_err() {  
    local exit_code=$?  
    echo "ERROR line $1: $2 (status=$exit_code)" >&2  
}  
  
trap 'report_err "$LINENO" "$BASH_COMMAND"' ERR
```

Use ERR traps to improve visibility, not to avoid writing clear logic.

4) Debugging recipes

Trace the whole script

```
bash -x script.sh
```

Useful when you do not want to edit the file just to turn tracing on.

Trace only one region

```
set -x  
## suspicious code here  
set +x
```

This keeps the rest of the script readable.

Customize trace lines with PS4

```
PS4='+ ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: '  
set -x
```

Helpful fields to include: - source file - line number - current function name

Use a DEBUG trap for short investigations

```
trap 'echo "line $LINENO -> $BASH_COMMAND"' DEBUG
```

The DEBUG trap runs **before** each simple command. It is powerful, but noisy.

Print variable state safely

```
printf 'user=%q\n' "$user"  
printf 'args=%q\n' "$@"
```

printf %q shows values in a shell-escaped form that makes spaces and special characters visible.

5) shellcheck cheatsheet

Run it on every script:

```
shellcheck script.sh
```

Common severity labels

- error — likely broken or dangerous
- warning — risky or confusing
- info — useful suggestion
- style — readability or convention improvement

Top 10 common warnings and what they usually mean

1. **Unquoted variable expansion**
Example: `rm $file`
Fix: `rm -- "$file"`
 2. **Useless use of cat**
Example: `cat file | grep text`
Fix: `grep text file`
 3. **Unsafe iteration with for x in \$(cat file)**
Fix with: `while IFS= read -r line; do ...; done < file`
 4. **Using [when [[is safer for bash string tests**
Especially relevant for pattern matching and `< / >` string comparisons.
 5. **Variables assigned but never used**
Remove dead code or use the value intentionally.
 6. **Arrays expanded without quotes**
Use `"${arr[@]}"`, not `${arr[@]}` when passing elements as separate arguments.
 7. **Masking failures in pipelines**
Add `set -o pipefail` if a left-side failure should matter.
 8. **Read without -r**
Use `read -r` to avoid backslash interpretation.
 9. **Globs or tests that may break on spaces**
Quote variables and prefer arrays.
 10. **Command substitutions used where direct commands are clearer**
Example: `for f in $(ls) is fragile; prefer globs or find ... -print0 plus a safe read loop.`
-

6) Common pitfalls

Pitfall: assuming set -e catches everything

It does not. Special cases include: - commands used as if tests - some commands in `&&` and `||` lists - some command substitutions and subshell contexts

If a failure is expected or meaningful, handle it explicitly.

Pitfall: reading \$? too late

This is wrong:

```
some_command
echo "done"
echo "$?"
```

The final `$?` now belongs to `echo`, not `some_command`.

Pitfall: trusting pipelines without pipefail

```
false | true
```

Without `pipefail`, that pipeline looks successful because the last command succeeded.

Pitfall: relying on unset variables becoming empty strings

With `set -u`, this becomes an error:

```
echo "$missing_var"
```

Use defaults when appropriate:

```
echo "${missing_var:-default}"
```

Pitfall: xtrace leaking secrets

Avoid tracing around commands that expose tokens, passwords, or private data.

Pitfall: changing IFS and then forgetting to quote

Changing IFS can reduce accidental splitting, but quoting is still the primary defense.

7) Suggested checklist before shipping a script

- Add a bash shebang
- Decide whether `set -euo pipefail` is appropriate
- Validate required arguments early
- Check external dependencies with `command -v`
- Quote variable expansions
- Use arrays or `while IFS= read -r` for safe iteration
- Add clear error messages with `die()`
- Use `shellcheck`
- Test with good input and bad input
- Run with `bash -n` and, when needed, `bash -x`

Module 11 — Advanced Patterns

Module 11 Notes — Advanced Bash

Printable reference for advanced patterns in bash scripts.

1) getopt's template

```
#!/bin/bash
verbose=false
file=""

usage() {
    echo "Usage: $0 [-h] [-v] [-f FILE]"
}

while getopt ":hv:f:" opt; do
    case "$opt" in
        h) usage; exit 0 ;;
        v) verbose=true ;;
        f) file="$OPTARG" ;;
        :) echo "Error: -$OPTARG needs a value" >&2; exit 1 ;;
        \?) echo "Error: invalid option -$OPTARG" >&2; exit 1 ;;
    esac
done
```

```
done
shift $((OPTIND - 1))
```

Key reminders

- getopt is for **short options**.
 - A trailing : means the option requires a value.
 - A leading : enables custom missing-argument handling.
 - Use shift \$((OPTIND - 1)) if you still need positional arguments afterward.
-

2) Long-option pattern

```
while [[ $# -gt 0 ]]; do
  case "$1" in
    --help)
      usage
      exit 0
      ;;
    --file)
      file="$2"
      shift 2
      ;;
    --verbose)
      verbose=true
      shift
      ;;
    *)
      echo "Unknown option: $1" >&2
      exit 1
      ;;
  esac
done
```

Notes

- Bash does not provide native long-option parsing through getopt.
 - Use shift for flags and shift 2 for --key value pairs.
 - Validate that \$2 exists before reading it for value-taking options.
-

3) Here-doc forms

Form	Interpolation?	Common use
<<EOF	Yes	Generate configs, JSON, scripts
<<'EOF'	No	Literal templates, examples
<<-EOF	Yes, strips leading tabs	Nicely indented here-docs
<<-'EOF'	No, strips leading tabs	Nicely indented literal blocks

Examples

```
cat <<EOF
user=$USER
home=$HOME
EOF

cat <<'EOF'
user=$USER
```

```
home=$HOME
EOF
```

4) Arithmetic forms

Form	What it does	Example
<code>\$(())</code>	Expands to a numeric value	<code>sum=\$((a + b))</code>
<code>(())</code>	Runs an arithmetic command	<code>((count++))</code>
<code>let</code>	Older arithmetic command form	<code>let "x = 5 * 2"</code>

Important rules

- Bash arithmetic is **integer only**.
- `5 / 2` becomes `2`.
- Use `awk` or `bc` for floating-point math.

```
x=$((5 / 2))
((count += 1))
let "area = width * height"
awk 'BEGIN { print 5 / 2 }'
```

5) Sourcing and libraries

```
## lib.sh
say_hi() {
    echo "hi"
}

main() {
    say_hi
}

[[ "${BASH_SOURCE[0]}" == "${0}" ]] && main "$@"
```

Notes

- `source lib.sh` runs the file in the **current shell**.
 - `./lib.sh` runs it in a **new process**.
 - `BASH_SOURCE[0]` refers to the file being read.
 - The guard prevents `main` from running when the file is sourced.
-

6) Common pitfalls

1. Trying to parse `--long` options directly with `getopts`
2. Forgetting `shift $((OPTIND - 1))`
3. Expecting floating-point math from `(( ))`, `$( ( ))`, or `let`
4. Using `<<EOF` when you meant to keep variables literal
5. Sourcing a file that auto-runs `main`
6. Forgetting to quote variable expansions in normal shell code